

RISC Principles and ARM Architecture

RISC Design Philosophy, Load/Store Model, AArch32 Programmer-Visible State, and Core Instruction Classes

Computer Architecture & Organization | Topic 2

Topic 2 Technical Roadmap

Topic Objective

Analyze the architectural philosophy of Reduced Instruction Set Computers (RISC) and examine how ARM32 / AArch32 realizes this model via registers, control flags, and instruction classes.

- **Section 1:** RISC Design Principles and the Load/Store Paradigm
- **Section 2:** CISC vs. RISC Architectural Trade-Offs
- **Section 3:** ARM Architectural Overview and Execution States
- **Section 4:** Registers and Programmer-Visible State (r0–r15, cpsr)
- **Section 5:** Core Instruction Classes: Data Processing, Memory, and Control Flow
- **Section 6:** Topic Summary and Worked Self-Test Exercises

Section 01

RISC Design Principles & the Load/Store Paradigm

Foundational RISC Design Principles

Reduced Instruction Set Computers (RISC) optimize the hardware-software interface by prioritizing simplicity and regularity:

- **Simpler Control Circuitry:** Regular instruction formats and orthogonal opcodes drastically reduce decode and control unit complexity.
- **Smaller Silicon Footprint:** Eliminating complex microcoded multi-cycle instructions frees die area for larger register files and caches.
- **Lower Energy Consumption:** Streamlined decoding reduces gate switching activity, yielding high performance-per-watt.
- **Predictable Pipelining:** Uniform instruction lengths ensure smooth, hazard-free pipeline flow.

The Load/Store Architectural Paradigm

Separation of Memory Access and Computation

In a pure RISC architecture, data processing instructions (arithmetic, logic) operate **exclusively on registers**. Memory is accessed strictly through dedicated **load** and **store** instructions.

Arithmetic Isolation

- ALU inputs must reside in the register file.
- ALU outputs are committed to a register.
- No arithmetic instruction can target a memory address directly.

Explicit Memory Staging

- `ldr`: Reads data from memory into a register.
- `str`: Writes data from a register into memory.
- Simplifies pipeline address generation and memory stages.

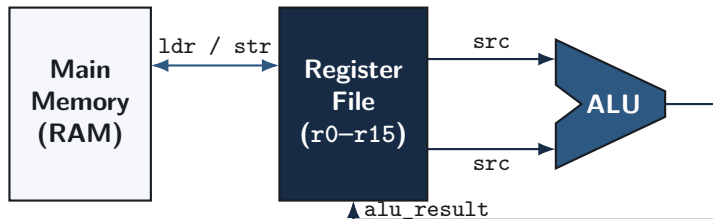
Executing Memory Mutation in RISC

To mutate a variable stored in RAM (e.g., $x = x + 5$), a RISC processor strictly sequences three distinct operations:

- ❶ **Load Stage:** Read the variable from memory address `[r0]` into working register `r1`.
- ❷ **Compute Stage:** Perform register-to-register addition using immediate value `#5`.
- ❸ **Store Stage:** Write modified value in `r1` back to memory address `[r0]`.

```
ldr r1, [r0]      // 1. Load x from RAM into register r1
add r1, r1, #5     // 2. Perform arithmetic within
register file
str r1, [r0]      // 3. Write updated result back to RAM
```

Visualizing the Load/Store Isolation



- The ALU is completely decoupled from the memory bus.
- Memory interactions occur exclusively across the register file boundary.

CISC versus RISC

Architectural Trade-Offs

Complex Instruction Set Computers (CISC)

CISC architectures maximize work accomplished per individual instruction:

- **Rich, Multi-Step Operations:** A single instruction combines memory fetching, arithmetic calculation, and memory write-back.
- **Direct Memory Arithmetic:** Instructions can operate directly on operands residing in memory (e.g., `add [eax], ebx`).
- **Variable-Length Formats:** Instruction encodings range from 1 to 15 bytes, requiring complex multi-stage decoders.
- **Dense Static Code:** High semantic density minimizes program size in RAM.

Design Penalty

Heavy silicon area and energy cost allocated to instruction decoding and complex hardware microcode control paths.

Code Comparison: Implementing $x += 5$

Consider incrementing memory variable x by immediate constant 5:

CISC Implementation (x86)

- **Instruction:** `add DWORD PTR [x], 5`
- Single variable-length instruction.
- CPU decodes operand, issues memory read, performs ALU add, and writes to memory in microcode.
- Compact binary size, but complex execution timing.

RISC Implementation (ARM32)

- **Instruction Stream:**
 - `ldr r1, [x]`
 - `add r1, r1, #5`
 - `str r1, [x]`
- Three fixed 32-bit instructions.
- Higher instruction count, but simple 1-cycle decode and pipeline predictability.

CISC vs. RISC Trade-Off Matrix

Dimension	CISC (e.g., x86-64)	RISC (e.g., ARM, RISC-V)
Instruction Size	Variable (1 to 15 bytes)	Fixed (typically 32-bit)
Memory Access	Combined with computation	Strict Load/Store only
Addressing Modes	Many, complex modes	Few, regular modes
Decode Complexity	High (requires decode ROM/ μ ops)	Low (hardwired, orthogonal fields)
Register Count	Fewer visible registers (8–16)	Larger register files (16–32+)
Pipelining	Difficult due to multi-cycle instructions	Highly efficient and deterministic

ARM Architectural Overview and Execution States

ARM Architecture Overview

ARM is a world-standard RISC instruction set architecture family based on regularity, register-centric computation, and explicit memory access:

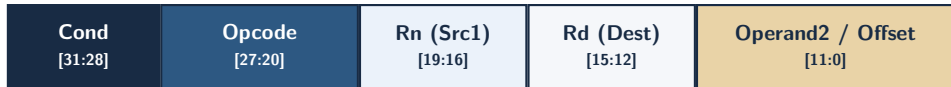
- Developed by Arm Ltd.; licensed to semiconductor designers worldwide (Apple, Qualcomm, AWS, NXP).
- **Target Paradigm:** High computational throughput per milliwatt, ideal for battery-constrained mobile devices and dense cloud datacenters.
- **Course Baseline:** We utilize **ARM32 / AArch32** as our primary model for assembly analysis, register behavior, and microarchitectural design.

Architectural Contract vs. Microarchitecture

The ARM ISA guarantees bit-exact software execution across cores ranging from simple in-order microcontrollers to 8-wide superscalar engines.

ARM Fixed-Width 32-Bit Instruction Encoding

In standard ARM state, every instruction is encoded into an exact **32-bit word**:



- **Simplified Alignment:** Instructions always align on 4-byte boundaries, eliminating unaligned instruction fetch penalties.
- **Consistent Field Placement:** Source register (`rn`) and destination register (`rd`) specifiers occupy identical bit locations across instruction families.
- **Datapath Reuse:** Common field placement minimizes multiplexer routing in the decode stage.

AArch32 Execution States: ARM vs. Thumb

The AArch32 architecture supports two primary operational instruction states:

ARM State

- Fixed-width 32-bit encodings.
- Full architectural feature access: hardware conditional execution on all instructions.
- Maximizes processing performance and ALU flexibility.

Thumb / Thumb-2 State

- Variable mixture of 16-bit and 32-bit encodings.
- Compresses code size by up to 35% for memory-constrained embedded ROM/Flash.
- Operates on the exact same register set.

Architectural Commonality

Both states share the underlying 32-bit register model (r0–r15) and status flags (cpsr).

Registers and Programmer-Visible State

ARM32 Programmer-Visible Register File

AArch32 exposes **16 visible 32-bit general-purpose registers** (r0–r15) plus the cpsr:

Register	Architectural Role	Standard Software Convention
r0–r3	General Purpose	Function argument inputs / return values
r4–r11	General Purpose	Callee-saved local variables
r12 (ip)	General Purpose	Intra-procedure call scratch register
r13 (sp)	Stack Pointer	Holds memory address of the current stack frame
r14 (lr)	Link Register	Holds return address for function calls (b1)
r15 (pc)	Program Counter	Holds the memory address of the executing instruction

The Program Counter (r15 / pc)

The **Program Counter** tracks the execution stream in memory:

- In **AArch32**, pc is architecturally mapped directly to r15.
- Software can read r15 or write to it directly using standard data instructions (e.g., `mov pc, lr` to return from a function).
- **Pipelining Offset**: Reading pc in ARM state returns current instruction address + 8 due to the 3-stage fetch-decode-execute pipeline prefetching.

Important Evolution: AArch64 Architectural Difference

In 64-bit ARM (**AArch64**), pc is **not** a general-purpose register. It cannot be targeted by arithmetic instructions and can only be updated through dedicated branch/jump operations.

Current Program Status Register (CPSR)

The **CPSR** holds runtime system status, processor mode bits, and arithmetic condition flags:

N [31]	Z [30]	C [29]	V [28]	Reserved / Control [27:8]	I, F [7:6]	Mode [4:0]
-----------	-----------	-----------	-----------	------------------------------	---------------	---------------

The Arithmetic Condition Flags (NZCV)

- **N (Negative)**: Set if the result bit 31 is 1 (negative signed value).
- **Z (Zero)**: Set if the result of the operation equals 0.
- **C (Carry)**: Set if an unsigned operation generated an arithmetic carry-out or non-borrow.
- **V (oVerflow)**: Set if a signed operation exceeded 32-bit 2's complement range.

Complete ARM Architectural State

Formal Definition

Architectural State comprises all hardware register contents and memory state that must be preserved to guarantee bit-exact program resumption.

Programmer-Visible Components

- 16 General Registers: r0–r15.
- Dedicated Roles: sp, lr, pc.
- Status State: cpsr (NZCV flags).
- Addressable memory space (RAM).

State Transitions

- Instructions execute by transitioning state:

$$\text{State}_{t+1} \leftarrow f(\text{State}_t, \text{Instruction})$$

- Microarchitectural structures (caches, pipelines) change physically but remain invisible.

Core AArch32 Instruction Classes

AArch32 Instruction Categories

Every operation in the ARM ISA partitions into one of three core functional classes:

1. Data Processing

- Arithmetic & logic.
- Register-only inputs.
- Optional flag updates.
- Examples: add, sub, mul, mov, cmp.

2. Memory Access

- Transfers between RAM and registers.
- Base register addressing.
- Examples: ldr, str, ldrb, strb.

3. Control Flow

- Branching and jumps.
- Direct mutation of pc.
- Conditional execution.
- Examples: b, beq, bne, blt, bl.

Class 1: Data Processing Instructions

Data processing instructions compute mathematical or logical operations within registers:

Instruction Syntax	Semantic Operation	Architectural Effect
<code>add r3, r1, r2</code>	Addition	$r3 \leftarrow r1 + r2$
<code>sub r3, r3, r2</code>	Subtraction	$r3 \leftarrow r3 - r2$
<code>mul r3, r1, r2</code>	Multiplication	$r3 \leftarrow r1 \times r2$
<code>mov r3, r4</code>	Register Transfer	$r3 \leftarrow r4$
<code>and r0, r1, r2</code>	Bitwise Logical AND	$r0 \leftarrow r1 \text{ AND } r2$
<code>orr r0, r1, r2</code>	Bitwise Logical OR	$r0 \leftarrow r1 \text{ OR } r2$

Immediate Constant Variant

Operands can be numerical literals: `add r1, r1, #5` adds constant 5 to register r1.

Class 2: Memory Access Instructions

Memory access instructions transfer words or bytes between RAM and the CPU register file:

```
ldr r1, [r2]    // Load 32-bit word at memory address in r2
str r5, [r3]    // Store 32-bit word from register r5 into
                // memory at address in r3
ldr r1, [r2, #4] // Base-plus-offset: Load from address
                // (r2 + 4) into r1
```

Bracket Notation [r2]

Brackets represent **memory dereferencing**. The register inside brackets contains the memory address pointer; the operation accesses the memory cell at that numerical address.

Comparison Operations and the `cmp` Instruction

Conditional decision-making begins with explicit comparison instructions:

`cmp r0, r1` (Compare Operands)

- Performs internal subtraction: $\text{Temporary} \leftarrow r0 - r1$.
- **Discards Result:** The numerical difference is not stored in any register.
- **Updates CPSR:** Evaluates and sets condition flags (**N**, **Z**, **C**, **V**) based on the subtraction.

If `r0 == r1`

$r0 - r1 = 0 \rightarrow$ **Z flag set to 1.**

If `r0 < r1`

$r0 - r1 < 0 \rightarrow$ **N flag set to 1.**

Class 3: Control Flow and Branching

Branch instructions alter program flow by loading new target addresses into the pc:

Unconditional Branch

`b target_label`

- Forces execution to diverge unconditionally.
- Overwrites pc with the target address.
- Equivalent to `jmp` in x86 assembly.

Conditional Branches

`b<cond> target_label`

- Evaluates NZCV flags in `cpsr`.
- If condition holds true $\rightarrow$ update pc.
- If false $\rightarrow$ fall through sequentially ($PC \leftarrow PC + 4$).

Mnemonic	Meaning	Tested Flag Condition
<code>beq</code>	Branch if Equal	$Z == 1$
<code>bne</code>	Branch if Not Equal	$Z == 0$
<code>blt</code>	Branch if Less Than (Signed)	$N \neq V$
<code>bgt</code>	Branch if Greater Than (Signed)	$(Z == 0) \text{ and } (N == V)$

Assembly Decisions: The Compare-and-Branch Pattern

High-level if-else statements are implemented via a 2-step assembly pattern:

Step 1: Evaluate Condition

```
cmp r1, r2
```

- Subtracts r2 from r1 internally.
- Updates **NZCV** flags in cpsr.

Step 2: Test Flags & Branch

```
beq equal_label
```

- Evaluates flag state (e.g., $Z == 1$).
- If true → updates pc to label.

Complete Conditional Pattern

```
cmp r1, r2           // 1. Compare r1 and r2
beq equal_path        // 2. If r1 == r2, branch to equal_path
sub r0, r1, r2        // Fall-through: executed if r1 != r2
b end_block           // Bypass true-block
equal_path:
    add r0, r1, r2    // True-block: executed if r1 == r2
```

Labels as Memory Aliases

What is an Assembly Label?

An assembly label (e.g., `target:`) is a human-readable alias representing the exact numerical **memory address** where subsequent instructions reside.

Assembly Source Code:

```
b compute_sum          // Branch
mov r0, #0
compute_sum:           // Label
add r0, r1, r2          // Target
```

Hardware Memory View:

Address	Machine Instruction
0x0x00008000	b +0x8 (0x0x00008008)
0x0x00008004	mov r0, #0
0x0x00008008	add r0, r1, r2

Execution Safety

The branch target must resolve to a valid instruction memory address to prevent invalid opcode crashes.

Summary & Self-Test

Topic 2 Comprehensive Summary

- **RISC Design Philosophy:** Emphasizes simple control circuitry, regular instruction formats, fixed-width encodings, and high energy efficiency.
- **Load/Store Architecture:** Strict isolation of memory access (`ldr`, `str`) from computation (`add`, `sub`).
- **AArch32 Register File:** 16 visible 32-bit registers (`r0–r15`) where `r13` is `sp`, `r14` is `lr`, and `r15` is `pc`.
- **Program Counter (`pc`):** In AArch32, `pc` is accessible as `r15` (with +8 pipeline offset); in AArch64, `pc` is separated from general registers.
- **Condition Flags (CPSR):** The `cmp` instruction updates **N**, **Z**, **C**, **V** flags to drive conditional branches (`beq`, `bne`, `blt`).
- **Instruction Classes:** All ARM instructions partition into Data Processing, Memory Access, or Control Flow.

Self-Test: Questions 1 & 2

Question 1: RISC vs. CISC Principles

What are the main architectural principles that distinguish RISC from CISC? Why does simpler instruction decoding reduce processor power consumption?

Question 2: The Load/Store Sequence

How does the ARM32 instruction sequence below implement an update to a memory variable? Why can't ARM perform this operation in a single arithmetic instruction?

```
ldr r1, [r0]    →    add r1, r1, #5    →    str r1, [r0]
```

Self-Test: Solutions 1 & 2

Solution 1

RISC uses fixed-width instructions, a load/store model, and regular operand fields. CISC uses variable-length encodings and direct memory arithmetic. Simpler decoding reduces silicon gate count and dynamic switching activity ($P \propto C \cdot V^2 \cdot f$), lowering power draw.

Solution 2

The sequence explicitly stages memory mutation: `ldr` loads data into `r1`, `add` computes in the ALU, and `str` writes back to RAM. ARM cannot combine these because the ALU datapath is isolated from memory buses, adhering to the pure RISC load/store principle.

Self-Test: Questions 3 & 4

Question 3: ARM32 Registers and Special Roles

Which ARM32 registers are general-purpose operands, and what special architectural roles are assigned to r13, r14, and r15? How is pc exposed differently in AArch64?

Question 4: Instruction Classes

What are the three core instruction classes in AArch32? Give one specific instruction example for each class.

Self-Test: Solutions 3 & 4

Solution 3

- r0–r12: General-purpose registers for operands and variables.
- r13 (sp): Stack Pointer; r14 (lr): Link Register; r15 (pc): Program Counter.
- In AArch64, the pc is **not** a general-purpose register and cannot be accessed as a numbered register operand.

Solution 4

- ① **Data Processing:** `add r3, r1, r2` (computes within registers).
- ② **Memory Access:** `ldr r1, [r2]` (loads word from memory into register).
- ③ **Control Flow:** `beq target` (conditionally branches by updating pc).

Self-Test: Question 5 & Solution

Question 5: Compare-and-Branch Execution

How do `cmp` and a conditional branch such as `blt` work together to implement high-level decision making? What register stores the intermediate outcome?

Self-Test: Question 5 & Solution

Question 5: Compare-and-Branch Execution

How do `cmp` and a conditional branch such as `blt` work together to implement high-level decision making? What register stores the intermediate outcome?

Solution 5

- `cmp r0, r1` performs an internal subtraction ($r0 - r1$), discards the numeric result, and updates the **N, Z, C, V** flags in the **CPSR** (Current Program Status Register).
- `blt label` subsequently inspects the flags ($N \neq V$) in the `cpsr`. If true, it loads the label address into `pc`; otherwise, it falls through to the next instruction.
- Intermediate comparison outcome is stored in the **CPSR**.